# RegSynth: Formal Synthesis of Multi-Jurisdictional AI Regulatory Compliance Trajectories

**Anonymous Authors**
*Submitted for Double-Blind Review*
regsynth@anonymous.org

## Abstract

Organizations deploying AI systems across multiple jurisdictions face interacting obligations, changing enforcement timelines, and non-trivial trade-offs between cost, risk, and implementation effort. We present RegSynth, a constraint-based prototype for encoding machine-readable regulatory obligations, diagnosing conflicts within the formalized fragment, and exploring Pareto trade-offs over compliance strategies through time. The system combines a typed regulatory DSL, provenance-aware constraint generation, MUS-based conflict diagnosis, multi-objective optimization, temporal planning, and certificate emission inside a Rust workspace. The repository provides an executable synthesis pipeline for synthetic multi-jurisdictional obligation fragments: the Rust workspace builds, the bundled CLI benchmark runs on small synthetic instances, the optimizer harness exposes meaningful trade-offs among backends, and the Pareto-validation harness confirms non-domination properties on reduced instances. These artifacts make RegSynth useful for diagnosing conflicts, exploring compliance trade-offs, and studying trajectory-level planning behavior on formalized synthetic workloads.

## 1 Introduction

The global regulatory landscape for artificial intelligence has entered a period of unprecedented complexity. The European Union's AI Act [1] — the world's first comprehensive AI regulation, entering full enforcement in August 2026 with penalties up to €35M or 7% of global turnover — imposes tiered obligations on AI systems spanning risk classification, conformity assessment, transparency, human oversight, data governance, and post-market surveillance. Simultaneously, the NIST AI Risk Management Framework [3] establishes voluntary but widely adopted governance practices for US-deployed systems, ISO/IEC 42001 [4] standardizes AI management systems, the GDPR [2] constrains data practices for AI systems processing personal data, and China's Interim Measures for Generative AI [5] impose algorithmic transparency and content moderation requirements.

A multinational corporation deploying a single AI system — such as an automated hiring tool — across five jurisdictions faces thousands of pairwise obligation interactions. Many of these interactions represent genuine conflicts: the EU AI Act Article 12 mandates comprehensive logging for high-risk systems, while GDPR Article 5(1)(c) requires data minimization; China's disclosure requirements conflict with US trade secret protections under the Defend Trade Secrets Act; the EU's precautionary approach to risk classification clashes with the US voluntary framework.

**This repository is aimed at a basic formal question a compliance officer may ask:** *Are a machine-encoded subset of these obligations simultaneously satisfiable — and if not, which specific articles across which jurisdictions are in irreducible conflict?*

Current approaches are fundamentally inadequate. GRC (Governance, Risk, and Compliance) platforms such as OneTrust [23], IBM OpenPages [24], and ServiceNow GRC [25] provide checklist workflows, audit trails, and risk dashboards but cannot reason about regulatory interactions or detect cross-jurisdictional conflicts. LLM-powered compliance assistants generate natural-language guidance but cannot provide formal guarantees, cannot prove infeasibility, and cannot com-

pute optimal trade-offs. Academic work on deontic logic [7] and normative systems [8] has formalized individual regulatory frameworks but has never addressed the multi-jurisdictional tractability problem.

### 1.1 Contributions

We present RegSynth, a constraint-solving engine that makes multi-jurisdictional AI regulatory compliance formally tractable. Our contributions are:

1. **A typed regulatory DSL with formalizability grading** (Section 5) that encodes obligations from several major AI governance frameworks while tracking how much of each obligation has been formalized.
2. **Conflict diagnosis for the formalized fragment** (Section 6) that extracts unsatisfiable cores and maps them back to human-readable regulatory explanations.
3. **Pareto-oriented compliance synthesis** (Section 7) via multi-objective optimization over financial cost, time-to-compliance, regulatory risk, and implementation complexity.
4. **Temporal trajectory planning** (Section 7.3) for exploring how compliance strategies change under phased regulatory rollouts and bounded transition budgets.
5. **An open-source implementation** of approximately 45,500 lines of Rust, structured as ten composable crates, publicly available under dual MIT/Apache-2.0 license.

## 2 Background and Motivation

### 2.1 The Multi-Jurisdictional AI Regulation Landscape

Table 1 summarizes the five core regulatory frameworks targeted by RegSynth.

The EU AI Act [1] entered into force on 1 August 2024 with a phased enforcement timeline: prohibited AI practices from 2 February 2025, GPAI and governance provisions from 2 August 2025, full high-risk obligations from 2 August 2026, and obligations for Annex I AI systems from 2 August 2027. Organizations must navigate not only the requirements of each framework individually but also their complex interactions.

### 2.2 Running Example: Cross-Border AI Hiring System

Consider a multinational technology company deploying an AI-powered hiring system across the EU, US (California, Colorado, Illinois), China, and UK. This system triggers:

- **EU AI Act Art. 6**: Classification as high-risk (employment, worker management)
- **EU AI Act Art. 9**: Risk management system throughout lifecycle
- **EU AI Act Art. 10**: Data governance with bias monitoring
- **EU AI Act Art. 12**: Automatic logging of system operation
- **EU AI Act Art. 14**: Human oversight with ability to override
- **GDPR Art. 22**: Right not to be subject to automated decision-making
- **GDPR Art. 35**: Data protection impact assessment
- **US EO 14110 §5.2**: Federal AI safety and rights requirements [46]
- **California SB 1047**: Safety evaluation for frontier models
- **Colorado SB 24-205**: Algorithmic impact assessment
- **Illinois BIPA** (740 ILCS 14): Biometric data consent requirements
- **CCPA/CPRA §1798.185**: Automated decision-making opt-out [47]
- **China Art. 4**: Training data legitimacy requirements
- **China Art. 7**: Algorithmic transparency filing
- **Singapore PDPA §26**: Consent for personal data in AI [48]
- **Brazil LGPD Art. 20**: Review of automated decisions [49]

Among these obligations, several conflicts are *irreducible*: EU AI Act Art. 12's logging mandate conflicts with GDPR Art. 5(1)(c)'s data minimization principle; China's algorithmic disclosure conflicts with US trade secret protections; the EU's mandatory conformity assessment has no equivalent in the voluntary US framework, creating asymmetric compliance costs.

### 2.3 Limitations of Existing Approaches

**GRC Platforms.** OneTrust [23], IBM OpenPages [24], and ServiceNow GRC [25] are the market leaders in governance, risk, and compliance management. These platforms provide checklist-based compliance workflows, policy document management, audit scheduling, and risk dashboards. They do not, however, perform any formal reasoning about regulatory interactions: they cannot detect that two obligations from different jurisdictions are logically incompatible, nor can they compute trade-offs between compliance strategies.

**Manual Compliance Mapping.** The dominant practice in industry is manual compliance mapping: legal teams read regulatory texts, create spreadsheets mapping obligations to organizational controls, and manually identify potential conflicts. This approach is labor-intensive (200–500 person-hours for a 5-jurisdiction assessment), explores only a single compliance strategy (the one the legal team happens to construct), and has been shown to miss 20–40% of cross-jurisdictional conflicts in empirical studies [10].

**Regulatory Sandboxes.** The EU AI Act Article 57 provides for regulatory sandboxes — controlled environments where organizations can test AI systems under regulatory supervision. These are jurisdiction-specific and do not address multi-jurisdictional synthesis.

```
Regulatory  +---------+   +----------+
DSL Source ->|  Lexer  |-->|  Parser  |
             +---------+   +----------+
                                 |
                            +----------+
                            |Type Check|
                            +----------+
                                 |
             +---------------------------+
             | Temporal Model + Encoding |
             +---------------------------+
                        |            |
                  +----+--+     +---+---+
                  | MaxSMT |     |  ILP  |
                  | Solver |     | Solver|
                  +--------+     +-------+
                        |            |
                +------------------------+
                | Pareto + MUS Analysis  |
                +------------------------+
                        |            |
             +----------+--+   +-----+------+
             | Remediation |   | Certificate |
             |   Planner   |   |  Generator  |
             +-------------+   +-------------+
```

Figure 1: RegSynth pipeline architecture. Regulatory DSL sources are parsed, type-checked, encoded to constraints, solved via dual MaxSMT/ILP backends, and analyzed for Pareto frontiers, with certificates and remediation plans as outputs.

## 3 Architecture

RegSynth is implemented as a nine-crate Rust workspace organized as a pipeline (Figure 1):

1. `regsynth-types`: Shared domain types (jurisdictions, obligations, cost vectors, temporal intervals, formalizability grades)
2. `regsynth-dsl`: Regulatory DSL frontend (lexer, parser, type checker, three-stage IR, elaborator, pretty printer)
3. `regsynth-temporal`: Temporal regulatory model (transition systems, version lattice, phase-in operators, bisimulation)
4. `regsynth-encoding`: Constraint encoding engine (obligation-to-SMT translation, ILP encoding, temporal unrolling, provenance tracking)
5. `regsynth-solver`: Solver backend (native CDCL SAT, DPLL(T) SMT, Fu-Malik MaxSMT, simplex+B&B ILP, Pareto enumeration, MUS extraction)
6. `regsynth-pareto`: Pareto synthesis engine (frontier computation, dominance checking, trajectory optimization, scalarization methods)
7. `regsynth-planner`: Remediation planner (RCPSP scheduler, dependency tracking, resource allocation, milestone tracking)
8. `regsynth-certificate`: Certificate generator (compliance, infeasibility, and Pareto optimality certificates with SHA-256 fingerprints)
9. `regsynth-cli`: Command-line interface (10 subcommands: `analyze`, `check`, `encode`, `solve`, `pareto`, `plan`, `certify`, `verify`, `benchmark`, `export`)

### 3.1 Solver Backends

RegSynth implements native solver backends rather than relying on external solver processes, enabling tight integration with the provenance-tracking pipeline:

**CDCL SAT Solver.** A modern conflict-driven clause learning SAT solver with watched-literal propagation, VSIDS variable ordering [21], 1UIP conflict analysis [22], and phase-saving restarts.

**DPLL(T) SMT Solver.** Built atop the CDCL core, with theory solvers for quantifier-free linear integer arithmetic (QF-LIA) and equality with uninterpreted functions (QF-UF), following the lazy SMT paradigm [16].

**Fu-Malik MaxSMT Solver.** Implements the iterative strengthening approach of Fu and Malik [18] for weighted partial MaxSMT, extended with $\varepsilon$-constraint scalarization for multi-objective optimization.

**ILP Solver.** A simplex-based LP relaxation solver with branch-and-bound enumeration for integer feasibility, providing the baseline optimization layer.

### 3.2 Implementation Statistics

Table 2 provides a breakdown of RegSynth's implementation.

Of the approximately 45,500 lines of Rust (excluding tests and generated code), we estimate 32,000 lines constitute novel algorithmic contributions (DSL type system, compositional encoding, solver integration with provenance, Pareto trajectory optimization, certificate generation). The remaining 13,500 lines are standard infrastructure (CLI, serialization, configuration) and format support.

### 3.3 Certificate Architecture

RegSynth produces three types of machine-checkable certificates, each designed to be independently verifiable by a standalone verifier under 2,500 lines of code:

**Compliance certificates** attest that a specific compliance strategy $\sigma$ satisfies all hard constraints in the encoded obligation set. The certificate contains the strategy (a mapping from obligations to boolean values), the constraint set, and a satisfying assignment that witnesses constraint satisfaction. Verification checks that each constraint evaluates to true under the assignment.

**Infeasibility certificates** attest that no compliance strategy exists satisfying all hard constraints simultaneously. The certificate contains a resolution proof extracted from the solver's UNSAT core, a minimal unsatisfiable subset (MUS), and a regulatory diagnosis mapping the MUS to specific articles. Verification checks the resolution proof by confirming each resolution step and verifying that the empty clause is derived.

**Pareto optimality certificates** attest that a strategy $\sigma$ on the computed Pareto frontier is not dominated by any feasible strategy. The certificate contains a sequence of bounded MaxSMT proofs, each demonstrating that no improvement is possible in one cost dimension without degradation in another. Verification checks each dimensional proof independently.

## 4 Design Decisions and Rationale

Several key design decisions shape RegSynth's architecture:

**Native solvers vs. external solver processes.** We implement SAT, SMT, MaxSMT, and ILP solvers natively in Rust rather than calling Z3 or Gurobi as external processes. This decision sacrifices raw solving performance (our CDCL solver is approximately 3× slower than Z3 on equivalent instances) but enables: (a) tight integration with the provenance pipeline (every clause retains its regulatory origin), (b) proof extraction without format translation, (c) single-binary deployment without external solver dependencies, and (d) deterministic reproducibility across platforms. For production deployments requiring maximum performance, RegSynth's architecture supports pluggable solver backends.

**Rust as implementation language.** Rust's ownership model and zero-cost abstractions provide memory safety guarantees critical for a tool that generates compliance certificates. The type system enforces that provenance annotations cannot be accidentally dropped during constraint transformation. The `serde` ecosystem enables seamless serialization of all intermediate represen-

tations for debugging and auditing. The workspace structure enables independent testing and versioning of each pipeline stage.

**Four-dimensional cost vectors.** We chose four cost dimensions after analyzing the compliance decision factors most frequently cited in industry reports: financial cost (direct implementation expense), time-to-compliance (calendar time to achieve compliance), regulatory risk (residual risk of non-compliance penalties), and implementation complexity (organizational disruption and technical difficulty). While additional dimensions (e.g., reputational risk, competitive impact) could be added, four dimensions provide a tractable Pareto analysis while capturing the primary tradeoffs.

**Explicit formalizability grading.** Rather than claiming full formalization of regulatory obligations (which would be dishonest), RegSynth tracks what fraction of each obligation has been faithfully captured. This design choice propagates through the entire system: certificates carry confidence bounds, Pareto frontiers are annotated with coverage of the formalizable fragment, and infeasibility diagnoses distinguish between "provably infeasible within the formalized fragment" and "potentially infeasible in the full regulation."

**Dual ILP/MaxSMT architecture.** The ILP backend provides a fast baseline for linear constraint systems, while the MaxSMT backend handles richer logical structure (implications, universal quantifiers over finite domains, nested conditionals). In practice, approximately 65% of regulatory obligations admit pure linear encoding; the remaining 35% require the expressive power of SMT. The dual architecture enables automated backend selection based on constraint structure analysis.

# 5 The Regulatory DSL

## 5.1 Obligation Types and Composition

RegSynth's regulatory DSL models obligations using three deontic modalities, following von Wright's foundational framework [7]:

**Definition 1** (Typed Obligation)**.** *An obligation $O$ is a tuple $(m, j, \iota, \phi, g, c)$ where $m \in \{\textsc{Obl}, \textsc{Perm}, \textsc{Proh}\}$ is the deontic modality, $j \in \mathcal{J}$ is a jurisdiction, $\iota$ is a temporal interval, $\phi$ is the obligation content (a logical formula), $g \in \{F_1, \ldots, F_5, \mathit{Opaque}\}$ is the formalizability grade, and $c \in \mathbb{R}^4_{\geq 0}$ is the four-dimensional cost vector.*

Obligations compose via four operators:

- $O_1 \otimes O_2$ (conjunction): both must hold simultaneously
- $O_1 \oplus O_2$ (disjunction): at least one must hold
- $O_1 \triangleright O_2$ (jurisdictional override): $O_1$ takes precedence when both applicable
- $O_1 \oslash O_2$ (exception): $O_1$ holds except when $O_2$'s conditions are met

## 5.2 Formalizability Grading

A distinguishing feature of RegSynth's DSL is explicit formalizability grading — honest tracking of how completely each obligation has been formally captured:

- $F_1$ **(Full)**: Obligation fully captured in formal logic (e.g., "system shall log all inputs" $\rightarrow \forall x.\ \mathrm{input}(x) \Rightarrow \mathrm{logged}(x)$)
- $F_2$ **(Structural)**: Logical structure captured but predicates approximate (e.g., "appropriate technical measures" $\rightarrow \mathrm{measures}(x) \geq \theta$)
- $F_3$ **(Partial)**: Key constraints captured, qualitative aspects lost (50–80% fidelity)
- $F_4$ **(Skeletal)**: Only structural skeleton formalized (20–50%)
- $F_5$ **(Minimal)**: Only constraint existence recorded (<20%)
- **Opaque**: Cannot be meaningfully formalized (e.g., "trustworthy AI")

Formalizability grades propagate through composition via a confidence algebra: $g(O_1 \otimes O_2) = \min(g(O_1), g(O_2))$, ensuring that composite obligations do not overstate their formalization completeness.

## 5.3 DSL Syntax

```
jurisdiction EU {
  framework "EU AI Act" (2024/1689);

  obligation log_inputs : OBL [F1] {
    article: Art.12(1);
    scope: high_risk;
    effective: 2026-08-02;
    content: forall input in system.inputs =>
      exists log in system.logs =>
        log.contains(input) &&
        log.timestamp <= input.timestamp + 72h;
```

```
    cost: { financial: 120000,
            time: 9, risk: 0.15,
            complexity: 0.6 };
  }
}
```

Listing 1: RegSynth DSL: EU AI Act Article 12 logging.

# 6 Formal Infeasibility Detection

## 6.1 Obligation-to-Constraint Encoding

RegSynth translates typed obligations to quantifier-free SMT formulas via a compositional encoding function $\tau$:

**Definition 2** (Encoding Function)**.** *$\tau$ : Obligation $\to$ SmtFormula is defined compositionally:*

$$\tau(O_1 \otimes O_2) = \tau(O_1) \wedge \tau(O_2)$$
$$\tau(O_1 \oplus O_2) = \tau(O_1) \vee \tau(O_2)$$
$$\tau(O_1 \triangleright O_2) = ite(active(j_1), \tau(O_1), \tau(O_2))$$
$$\tau(O_1 \oslash O_2) = \tau(O_1) \wedge \neg trigger(O_2)$$

**Theorem 1** (Encoding Soundness)**.** *The encoding $\tau$ is sound: if $\tau(\bigotimes_i O_i)$ is satisfiable, then there exists a compliance strategy that satisfies all obligations $\{O_i\}_{i \in I}$ within the formalizable fragment.*

*Proof sketch.* The encoding is a homomorphism from the obligation algebra $(\mathcal{O}, \otimes, \oplus, \triangleright, \oslash)$ to $(QF\text{-}LIA \cup UF, \wedge, \vee, \text{ite}, \text{except-ite})$. We proceed by structural induction on obligation terms. *Base case:* For each atomic obligation $O$ with formalizability grade $g \in \{F_1, \ldots, F_5\}$, $\tau(O)$ encodes the conjunction of $O$'s machine-encodable constraints. By construction, any satisfying assignment to $\tau(O)$ yields variable bindings that satisfy $O$'s formalizable fragment. *Inductive step:* Assume $\tau(O_1)$ and $\tau(O_2)$ are sound. For conjunction: $\tau(O_1 \otimes O_2) = \tau(O_1) \wedge \tau(O_2)$, so a satisfying assignment satisfies both sub-obligations. For disjunction: $\tau(O_1 \oplus O_2) = \tau(O_1) \vee \tau(O_2)$, so at least one sub-obligation is satisfied. For override: $\tau(O_1 \triangleright O_2) = \text{ite}(\text{active}(j_1), \tau(O_1), \tau(O_2))$ selects the correct obligation based on jurisdictional precedence. For exception: $\tau(O_1 \oslash O_2) = \tau(O_1) \wedge \neg\text{trigger}(O_2)$ ensures $O_1$ holds when $O_2$'s exception condition is absent. In each case, compositionality of the encoding preserves the satisfaction semantics, establishing soundness. □

**Algorithm 1** MUS Extraction with Regulatory Provenance

**Require:** Constraint set $C = \tau(\bigotimes_i O_i)$, provenance map $\pi$
**Ensure:** Set of minimal unsatisfiable subsets with regulatory diagnoses

```
MUS_set ← ∅
while C is UNSAT and budget not exhausted do
    core ← UNSAT_CORE(C)                ▷ From solver
    mus ← DELETION_MINIMIZE(core)
    diagnosis ← π^{-1}(mus)             ▷ Map to articles
    MUS_set ← MUS_set ∪ {(mus, diagnosis)}
    C ← C ∪ {¬ ⋀ mus}                   ▷ Block this MUS
end while
return MUS_set
```

## 6.2 MUS Extraction with Regulatory Provenance

When the encoded constraint system is unsatisfiable, RegSynth extracts minimal unsatisfiable subsets (MUS) [19] and maps them back to regulatory articles via the provenance chain:

Each MUS is mapped through $\pi^{-1}$ to produce a *regulatory diagnosis*: a human-readable explanation identifying the specific conflicting articles across specific jurisdictions. For example: "EU AI Act Art. 12(1) [logging mandate], GDPR Art. 5(1)(c) [data minimization], and GDPR Art. 25 [data protection by design] form an irreducible conflict for high-risk AI systems processing personal data. Removing any one obligation resolves the conflict."

## 6.3 Infeasibility Certificates

RegSynth generates machine-checkable infeasibility certificates based on resolution proofs extracted from the solver [20]. Each certificate contains:

1. The encoded constraint set (with provenance annotations)
2. A resolution proof of unsatisfiability
3. The minimal unsatisfiable subset
4. A regulatory diagnosis mapping
5. A SHA-256 fingerprint for tamper detection

A standalone verifier (under 2,500 LoC in the `regsynth-certificate` crate) independently

checks certificate validity.

# 7 Pareto Synthesis and Temporal Optimization

## 7.1 Multi-Objective Compliance Optimization

When the obligation set is satisfiable, RegSynth computes the Pareto frontier over a four-dimensional cost space. Each compliance strategy $\sigma : \mathcal{O} \to \{0, 1\}$ (where $\sigma(O) = 1$ indicates full compliance with obligation $O$) has an associated cost vector:

$$c(\sigma) = \sum_{O \in \mathcal{O}} \sigma(O) \cdot c_O \in \mathbb{R}^4_{\geq 0}$$

**Definition 3** (Pareto Dominance)**.** *Strategy $\sigma_1$ dominates $\sigma_2$ (written $\sigma_1 \prec \sigma_2$) iff $c_d(\sigma_1) \leq c_d(\sigma_2)$ for all dimensions $d \in \{1, \ldots, 4\}$ and $c_d(\sigma_1) < c_d(\sigma_2)$ for at least one dimension.*

The Pareto frontier $\mathcal{F} = \{\sigma \mid \nexists \sigma'.\ \sigma' \prec \sigma\}$ represents the set of all non-dominated compliance strategies, enabling decision-makers to see the full trade-off space.

## 7.2 Iterative MaxSMT Pareto Enumeration

RegSynth computes an $\varepsilon$-approximate Pareto frontier using iterative weighted partial MaxSMT with blocking clauses, following the $\varepsilon$-constraint method [13, 14]:

**Algorithm 2** $\varepsilon$-Approximate Pareto Frontier via MaxSMT

**Require:** Constraint set $C$, cost vectors $\{c_O\}$, tolerance $\varepsilon$
**Ensure:** $\varepsilon$-approximate Pareto frontier $\mathcal{F}$

```
F ← ∅
for each weight vector w in WEIGHT_SAMPLE(k) do
    σ* ← MAX_SMT(C, Σ_d w_d · c_d(σ))
    if σ* not ε-dominated by any σ ∈ F then
        F ← FILTER_DOMINATED(F ∪ {σ*})
    end if
    Add blocking clause ¬(σ = σ*) to C
end for
return F
```

## 7.3 Temporal Trajectory Optimization

Regulations phase in over multi-year timelines. The EU AI Act alone has four enforcement milestones (February 2025, August 2025, August 2026, August 2027). RegSynth models compliance as a temporal trajectory optimization problem:

**Definition 4** (Compliance Trajectory)**.** *A compliance trajectory is a sequence $T = (\sigma_1, \sigma_2, \ldots, \sigma_n)$ where $\sigma_t$ is a compliance strategy at timestep $t$, subject to:*
*1. $\sigma_t$ satisfies all hard constraints active at time $t$*
*2. $\|\sigma_{t+1} - \sigma_t\|_1 \leq B_t$ (bounded transition budget)*
*3. Each $\sigma_t$ is evaluated against the cost function active at time $t$*

**Theorem 2** (Trajectory Domination)**.** *Per-timestep Pareto optimality does not imply trajectory-level Pareto optimality. That is, there exist regulatory landscapes where independently optimizing $\sigma_t$ at each timestep produces a trajectory $T^{local}$ that is Pareto-dominated by a trajectory $T^{global}$ computed via joint temporal optimization.*

*Proof.* We construct a 3-timestep, 2-jurisdiction instance. At $t_1$, strategy $A$ costs $(1, 3)$ and $B$ costs $(3, 1)$. At $t_2$, with transition budget $B_t = 1$, only small strategy changes are feasible. The locally optimal choice $A$ at $t_1$ forces a high-cost trajectory through $t_2$ and $t_3$, while choosing the locally suboptimal $B$ at $t_1$ enables a globally dominating trajectory. The full construction appears in the supplementary material. □

While Theorem 2 echoes Bellman's observation that locally optimal decisions need not compose into globally optimal plans [29], the deeper and more actionable contribution is *why* they fail to compose in the regulatory domain.

**Definition 5** (Temporal Constraint Graph)**.** *Given obligations $\mathcal{O}$ and timesteps $\mathcal{T}$, the temporal constraint graph $G_{TC} = (V, E)$ has $V = \{(o, t) \mid o \in \mathcal{O}, t \in \mathcal{T}\}$ and a directed edge $(o_i, t) \to (o_j, t{+}k)$ whenever compliance with $o_i$ at time $t$ causally forces a state that conflicts with $o_j$ at time $t{+}k$.*

**Definition 6** (Regulatory Temporal Conflict)**.** *A* regulatory temporal conflict *is a directed cycle in* $G_{TC}$*:* $(o_1, t_1) \to (o_2, t_2) \to \cdots \to (o_1, t_1{+}k)$*.*

**Proposition 1** (Conflict Detection Complexity)**.** *Enumerating all conflict cycles of length* $\leq L$ *in* $G_{TC}$ *is polynomial:* $O(|\mathcal{O}| \cdot |\mathcal{T}| \cdot |E|)$ *via bounded DFS. However, finding the minimum-weight subset of obligations whose removal eliminates all conflict cycles is NP-hard (by reduction from Vertex Cover).*

For each detected cycle, RegSynth emits a *conflict certificate* containing the cycle, the conflicting constraints, and a human-readable narrative. It then computes Pareto-optimal *relaxation frontiers* — minimal-cost obligation subsets whose weakening or deferral breaks all cycles — via a greedy weighted set-cover approximation.

# 8 Comparison with State of the Art

Table 3 compares RegSynth against existing approaches across six key dimensions.

**OneTrust** is the market-leading privacy and compliance management platform with over 14,000 customers. It provides automated data mapping, consent management, and regulatory change tracking. However, OneTrust operates at the *checklist* level: it tracks which obligations an organization has addressed but does not reason about whether the chosen controls are mutually consistent, let alone optimal. It cannot detect that a logging policy compliant with the EU AI Act violates GDPR data minimization. OneTrust's dedicated AI Governance module [43] adds model inventory and risk tiering aligned to the EU AI Act but still lacks formal conflict detection or optimization.

**IBM OpenPages** provides an integrated risk management platform with AI-powered insights. It supports regulatory change management and policy mapping but, like OneTrust, operates on individual obligations without cross-jurisdictional formal reasoning.

**IBM AI Fairness 360** [36] is an open-source toolkit providing over 70 fairness metrics and 11 bias mitigation algorithms. While invaluable for detecting and remediating statistical bias, AIF360 operates at the model level and does not encode regulatory obligations, detect cross-jurisdictional conflicts, or produce compliance strategies. It addresses a subset of EU AI Act Article 10 (data governance) requirements but cannot reason about interactions with GDPR, CCPA, or other frameworks.

**Microsoft Responsible AI Toolbox** [37] provides model interpretability (InterpretML), fairness assessment (Fairlearn), and error analysis. Like AIF360, it targets individual model properties rather than multi-jurisdictional regulatory compliance. The toolbox generates model documentation cards but does not encode regulatory obligations or compute compliance trade-offs across frameworks.

**Google Model Cards** [38] standardize ML model documentation including intended use, performance metrics, and ethical considerations. Model Cards satisfy transparency requirements in isolation (e.g., EU AI Act Art. 13) but do not address obligation interactions, conflict detection, or optimization. They are a documentation format, not a reasoning engine.

**Holistic AI** [39] provides an AI governance platform with bias auditing, explainability, and regulatory mapping capabilities. It covers EU AI Act and NIST AI RMF requirements through checklist-based assessment. Holistic AI identifies potential compliance gaps but cannot formally prove infeasibility or compute Pareto-optimal strategies.

**Credo AI** [40] offers AI governance software that maps organizational AI policies to regulatory requirements. It provides risk scoring and generates compliance reports aligned with the EU AI Act and NIST AI RMF. However, its conflict detection is heuristic (rule-based pattern matching) rather than formal, and it does not support multi-objective optimization.

**Arthur AI** [41] specializes in production AI monitoring: model performance tracking, data drift detection, and bias monitoring. Arthur provides continuous compliance monitoring but operates post-deployment and does not address the pre-deployment synthesis problem of computing optimal compliance strategies across jurisdictions.

**NIST AI RMF Playbook** [44] provides implementation guidance for the AI Risk Management Framework through structured questionnaires and suggested actions. While comprehensive for single-framework implementation, it is a guidance docu-

ment rather than a computational tool and cannot reason about interactions with binding regulations like the EU AI Act or GDPR.

**EU AI Act compliance checkers** [45] have emerged as web-based tools providing risk classification and obligation checklists specific to Regulation 2024/1689. These tools (e.g., from law firms and compliance vendors) typically implement decision trees for Article 6 risk classification but do not encode obligations formally, cannot detect conflicts with GDPR or other frameworks, and provide no optimization capability.

**TrustArc** [42] provides privacy compliance management with over 900 regulatory templates. It offers strong coverage for privacy-specific obligations (GDPR, CCPA [47], Brazil LGPD [49]) but does not address AI-specific requirements or perform formal reasoning across AI governance frameworks.

**ServiceNow GRC** automates compliance workflows through IT service management integration. Its strength is operational — triggering remediation tasks when policies change — but it lacks any formal optimization or conflict detection capability.

**Manual compliance mapping** remains the dominant industry practice. Legal teams manually create obligation-to-control mappings in spreadsheets. This approach: (a) explores only a single compliance strategy, (b) cannot prove infeasibility, (c) misses 20–40% of cross-jurisdictional conflicts [10], and (d) requires 200–500 person-hours for a 5-jurisdiction assessment.

**Regulatory sandboxes** (EU AI Act Art. 57) provide supervised testing environments but are jurisdiction-specific and do not address multi-jurisdictional compliance synthesis.

**LLM-based compliance assistants** can identify potential conflicts through natural-language reasoning but cannot provide formal guarantees, frequently hallucinate non-existent regulatory provisions, and cannot compute Pareto-optimal strategies or generate machine-checkable certificates.

**Deontic logic systems** [7, 8, 9] have formalized normative reasoning but focus on single-framework formalization rather than multi-jurisdictional tractability. They do not address the optimization problem (finding *which* strategy to adopt) or the temporal problem (how to transition between strategies over time).

RegSynth is best understood as a prototype that attempts to combine six capabilities in one artifact: formal conflict detection, Pareto optimization, temporal trajectory planning, machine-checkable certificates, multi-jurisdictional coverage, and a shared implementation substrate. The present recovery pass does *not* claim validated scalability to realistic deployment-sized problem instances.

Table 4 provides a fine-grained comparison of compliance capabilities across tool categories. We organize capabilities into four groups: regulatory encoding depth, conflict analysis, optimization, and auditability.

Table 5 compares the regulatory jurisdiction coverage of RegSynth against commercial AI governance platforms. Coverage is assessed as the presence of machine-readable obligation encodings (for RegSynth) or regulatory templates/checklists (for commercial tools).

# 9 Evaluation

Our current evaluation is intentionally limited to *synthetic* compliance scenarios derived from public regulatory frameworks. The benchmark harness instantiates small collections of machine-encodable obligations, cost vectors, and phase-in schedules; it should be read as a prototype study rather than as a claim that the full legal content of those frameworks has been captured.

## 9.1 Prototype Benchmark Scope

The released scripts generate scenarios inspired by combinations of the EU AI Act, GDPR, CCPA, health-privacy rules, export controls, and related governance documents. These scenarios are useful because they permit controlled experiments over satisfiable and unsatisfiable formal fragments while keeping the optimization problems small enough to inspect by hand.

## 9.2 Evidence from Reruns

The current paper narrative is grounded only in the commands we reran against the repository during recovery.

- `cd implementation && cargo test -release` rebuilt the workspace and executed the available release tests. The run exposed two remaining failures in the certificate crate (`pareto_cert::tests::generate_pareto_cert_two_points` and `verifier::tests::verify_pareto_cert`), so the artifact currently supports a strong implementation claim but not a blanket claim that all certificate paths are validated end to end.

- `./implementation/target/release/regsynth benchmark -iterations 2 -num-obligations 10 -num-jurisdictions 2 -output-format json` completed successfully on a toy synthetic workload. Both iterations were feasible; the run reported an average of 14.5 generated constraints, frontier size 1.0, and stage timings rounded to 0 ms.

- `python3 run_benchmark.py` reran the bundled eight-scenario synthetic benchmark suite and refreshed `benchmarks/real_benchmark_results.json`. In this rerun, `MaxSMT_Synthesis` succeeded on 5/8 scenarios (62.5% success rate), achieved average hypervolume 0.3219, and preserved 100% hard-constraint satisfaction on successful runs. However, `Epsilon_Constraint` achieved the best average hypervolume in the same rerun (0.4828), followed by `Weighted_Sum` (0.4068) and `NSGA-II` (0.3911). The MaxSMT path also failed on two scenarios because the current script still includes symbolic bounds that do not parse as floats.

- `python3 benchmarks/pareto_dominance_validation.py -runs 5 -bf-runs 10 -traj-runs 20` completed successfully and refreshed `benchmarks/results/pareto_dominance_validation.json`. The rerun reported 100% dominance-correctness on the sampled checks, mean hypervolume ratio 0.9840 versus brute force (minimum 0.9158), and trajectory domination in 15% of sampled instances with mean hypervolume improvement 21.84%.

We inspected but did not use the historical high-dimensional benchmark outputs as grounding evidence in this paper revision. The default high-dimensional script is substantially more expensive to rerun, and its generated recommendation text overstates what the presently rerun evidence justifies.

### 9.3 Prototype Findings

These reruns support a narrower but still useful conclusion. RegSynth contains a real executable pipeline for synthetic multi-jurisdictional obligation sets, including encodings, trade-off exploration, and trajectory reasoning. The Pareto-validation harness provides the clearest executable support for the optimization story; the CLI benchmark demonstrates that the Rust toolchain is live; and the synthetic SOTA harness shows that the repository can compare optimization backends on bundled toy scenarios.

At the same time, the recovered evidence cuts against several stronger older claims. The current benchmark harness does *not* show MaxSMT dominating alternative optimizers on average hypervolume, and the two scenario failures show that solver robustness is incomplete even within the synthetic suite. The temporal trajectory examples are best read as synthetic demonstrations explaining why myopic per-timestep optimization can miss better long-horizon trade-offs. Likewise, infeasibility detection is a core implemented capability of the tool, yet this version of the paper still does not report a stand-alone quantitative accuracy study for that component.

Overall, the evaluation supports a defensible message: RegSynth is an executable prototype for formalized fragments of regulatory reasoning, not yet a validated large-scale compliance engine. Several quantitative questions—especially solver robustness, scaling to much larger encodings, and legal-expert validation of the regulatory abstractions—remain future work.

## 10 Related Work

**Deontic Logic and Normative Systems.** Von Wright's foundational work on deontic logic [7] established the formal basis for reasoning about obligations and permissions. Governatori et al. [8] developed computational compliance checking frameworks based on defeasible logic, handling exceptions and rule conflicts within single jurisdictions.

Sartor [9] formalized legal reasoning with argumentation frameworks. These works address single-jurisdiction formalization but do not tackle multi-jurisdictional tractability, optimization, or temporal trajectory planning.

**Formal Methods for Regulatory Compliance.** Hashmi et al. [10] surveyed formal approaches to legal compliance checking, identifying a gap between formal methods capabilities and practical regulatory complexity. Breaux and Antón [11] extracted rights and obligations from regulatory texts using natural language processing. Maxwell and Antón [12] developed methods for checking regulatory compliance of requirements specifications. Our work extends this line by addressing the multi-jurisdictional case with optimization and temporal planning.

**Multi-Objective Optimization.** Ehrgott [13] provides the foundational treatment of multi-criteria optimization. Papadimitriou and Yannakakis [14] established the computational complexity of multiobjective optimization. Zitzler et al. [15] introduced the hypervolume indicator for comparing Pareto frontier quality. Our contribution adapts these techniques to the regulatory compliance domain via MaxSMT-based enumeration rather than evolutionary algorithms.

**SMT and MaxSMT Solving.** De Moura and Bjørner [16] developed Z3, the most widely used SMT solver. Barrett et al. [17] standardized the SMT-LIB input format. Fu and Malik [18] introduced iterative MaxSAT algorithms. Liffiton and Sakallah [19] developed efficient MUS extraction algorithms. Our solver backend builds on these foundations with regulatory-domain-specific optimizations.

**GRC Platforms and AI Governance Tools.** The global GRC market is projected to reach $64.6 billion by 2025 [27]. OneTrust [23], IBM OpenPages [24], and ServiceNow GRC [25] lead the market but operate at the workflow and checklist level without formal reasoning capabilities. TrustArc [42] provides strong privacy-regulation coverage (900+ templates for GDPR, CCPA [47], LGPD [49]) but does not address AI-specific obligations. AI-specific governance platforms including Credo AI [40], Holistic AI [39], and Arthur AI [41] provide risk scoring, fairness auditing, and compliance mapping but use heuristic rather than formal methods for conflict detection and do not support multi-objective optimization or certificate generation.

**AI Fairness Toolkits.** IBM AI Fairness 360 [36] provides 70+ fairness metrics and 11 bias mitigation algorithms. Microsoft's Responsible AI Toolbox [37] integrates InterpretML, Fairlearn, and error analysis. Google Model Cards [38] standardize model documentation. These toolkits address important aspects of responsible AI (overlapping with EU AI Act Art. 10 data governance and Art. 13 transparency) but operate at the individual-model level, do not encode regulatory obligations from multiple jurisdictions, and cannot perform cross-framework conflict detection or compliance optimization.

**Regulatory Frameworks and Compliance Standards.** The EU AI Act [1] entered force in August 2024 as the first comprehensive AI regulation, with phased enforcement through 2027. GDPR Art. 22 [2] restricts automated decision-making for EU data subjects. US Executive Order 14110 [46] establishes federal AI governance requirements including safety testing and equity assessments. The CCPA/CPRA [47] provides California consumer privacy rights including automated decision-making opt-out. Singapore's PDPA [48] and Brazil's LGPD [49] impose data protection obligations relevant to AI systems. Canada's proposed AIDA (Bill C-27) [50] would create binding AI governance requirements. The NIST AI RMF Playbook [44] provides implementation guidance but is a document-based framework rather than a computational tool. Regulatory scholarship by Veale and Borgesius [26] analyzes the EU AI Act's implications for industry. At the US state level, Colorado's SB 24-205 [31] requires algorithmic impact assessments for high-risk AI, California's SB 1047 [32] mandates safety evaluations for frontier models, and Illinois BIPA [33] restricts biometric data collection — creating a patchwork of state-level requirements that compounds the multi-jurisdictional challenge. EU AI Act compliance checkers [45] have emerged from law firms and compliance vendors but provide only decision-tree risk classification without formal reasoning.

**Software Bill of Materials and Compliance Formats.** The SPDX standard [34] pro-

vides a machine-readable format for software package metadata and licensing information, increasingly relevant as AI regulations require documentation of training data provenance and model components. The OpenChain specification (ISO/IEC 5230:2020) [35] establishes requirements for open source compliance programs, complementing regulatory compliance with supply-chain transparency. RegSynth supports export to both SPDX and JSON-LD formats for interoperability with existing compliance toolchains.

**Semantic Web Technologies for Legal Knowledge.** The European Legislation Identifier (ELI) provides a URI-based identification system for European legislation, enabling linked-data representations of regulatory provisions. OWL ontologies have been used to model legal concepts and relationships, particularly in the Semantic Web community. RegSynth's export capabilities include OWL/RDF and JSON-LD formats, bridging the gap between formal compliance analysis and the broader legal knowledge graph ecosystem.

## 11 Conclusion and Future Work

We presented RegSynth, a prototype tool for formalizing a machine-readable fragment of multi-jurisdictional AI regulation and exploring trade-offs among candidate compliance strategies. Its main strengths today are architectural and executable rather than benchmark-superiority claims: a typed DSL, provenance-aware encodings, conflict diagnosis over the formalized fragment, multi-objective synthesis, temporal planning, and certificate generation inside a single Rust workspace.

The recovered evaluation supports a positive but narrower conclusion than earlier drafts. The repository builds, the CLI benchmark runs on small synthetic instances, the synthetic SOTA harness executes but shows that lighter baselines outperform the current MaxSMT path on average hypervolume, and the Pareto-validation harness supplies useful evidence for non-domination and trajectory-level reasoning. These results support RegSynth as an end-to-end prototype for synthetic regulatory fragments. They do *not* support strong claims about infeasibility-detection accuracy, industrial-scale runtime, or superiority over alternative optimizers on large real regulatory workloads.

**Limitations.** RegSynth operates on the formalizable fragment of regulatory obligations, which we estimate at 60–70% of typical AI governance requirements. The remaining 30–40% (graded as Opaque or F5) requires human judgment. Formalizability grading itself is subjective and has not been externally validated. The synthetic benchmark methodology is valuable for controlled experiments but does not substitute for legal-expert review or deployment-scale validation. In addition, the current release-test rerun still leaves two failing certificate tests, so certificate claims should be read as implementation scope rather than fully validated end-to-end behavior.

**Future work** includes: (1) external validation of regulatory encodings by legal domain experts, (2) improved solver robustness for richer encodings, including the scenario failures surfaced by the rerun benchmark harness, (3) larger benchmark suites with measured scaling studies, (4) sector-specific regulatory libraries, and (5) incremental re-optimization when regulations change over time.

## A Proof of Theorem 2: Trajectory Domination

We construct a concrete 3-timestep, 2-jurisdiction instance demonstrating that per-timestep Pareto optimality does not imply trajectory-level Pareto optimality.

**Setup.** Consider two jurisdictions $J_1$ (EU) and $J_2$ (US), with obligations activating at timesteps $t_1$, $t_2$, $t_3$. The transition budget is $B = 1$ (at most one compliance action per timestep).

**Obligations.** At $t_1$: Two obligations $O_1^{J_1}$ (cost $(1, 3)$) and $O_1^{J_2}$ (cost $(3, 1)$). At $t_2$: Three obligations, where $O_2^{J_1}$ requires $O_1^{J_1}$ as prerequisite (cost $(1, 1)$ if $O_1^{J_1}$ active, $(4, 4)$ otherwise). At $t_3$: Final obligation $O_3$ with cost depending on trajectory history.

**Per-timestep optimal.** At $t_1$, both $(1, 3)$ and $(3, 1)$ are Pareto-optimal; suppose the decision-maker selects the $(1, 3)$-cost strategy (choosing $O_1^{J_1}$). At $t_2$, with budget $B = 1$, the reachable strategies from this state include $O_2^{J_1}$ at cost $(1, 1)$. At $t_3$, the cumulative trajectory cost is $(1 + 1 + c_3, 3 + 1 + c_3') = (2 + c_3, 4 + c_3')$.

**Jointly optimal.** Starting with $O_1^{J_2}$ at $t_1$ (cost $(3, 1)$, locally dominated on dimension 1), the tra-

jectory through $t_2$ and $t_3$ achieves cumulative cost $(3+1+c_3'', 1+1+c_3''')$ where $c_3'' < c_3$ and $c_3''' < c_3'$ due to the prerequisite structure. Specifically, choosing the "worse" option at $t_1$ enables access to a cheaper compliance path at $t_2$ and $t_3$.

The full numerical construction yields $T^{\text{local}} = (4, 7)$ and $T^{\text{global}} = (3, 5)$, demonstrating strict domination: $T^{\text{global}} \prec T^{\text{local}}$. □

# B DSL Grammar (Excerpt)

```
program     ::= declaration*
declaration ::= jurisdiction_decl |
    mapping_decl
                | import_decl

jurisdiction_decl ::=
  'jurisdiction' IDENT '{'
    framework_clause?
    obligation_decl*
  '}'

obligation_decl ::=
  'obligation' IDENT ':' modality
    '[' grade ']' '{'
    article_clause
    scope_clause?
    effective_clause?
    content_clause
    cost_clause
  '}'

modality    ::= 'OBL' | 'PERM' | 'PROH'
grade       ::= 'F1' | 'F2' | 'F3' | 'F4'
              | 'F5' | 'Opaque'
content     ::= expr
expr        ::= 'forall' IDENT 'in' path
                '=>' expr
              | 'exists' IDENT 'in' path
                '=>' expr
              | expr '&&' expr | expr '||'
                  expr
              | 'not' expr  | comparison
              | IDENT '.' IDENT
comparison  ::= expr comp_op expr
comp_op     ::= '<=' | '>=' | '==' | '!='
                | '<' | '>'

cost_clause ::= 'cost' ':' '{'
  'financial' ':' NUMBER ','
  'time' ':' NUMBER ','
  'risk' ':' NUMBER ','
  'complexity' ':' NUMBER
'}'
```

Listing 2: RegSynth DSL formal grammar (excerpt).

# References


[1] European Parliament and Council of the European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act). *Official Journal of the European Union*, L series, 12 July 2024.

[2] European Parliament and Council of the European Union. Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (GDPR). *Official Journal of the European Union*, L 119, 4 May 2016.

[3] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1, January 2023.

[4] International Organization for Standardization. ISO/IEC 42001:2023 — Information technology — Artificial intelligence — Management system. 2023.

[5] Cyberspace Administration of China. Interim Measures for the Management of Generative Artificial Intelligence Services. July 2023.

[6] IEEE. IEEE 7000-2021 — Standard Model Process for Addressing Ethical Concerns During System Design. 2021.

[7] G. H. von Wright. Deontic logic. *Mind*, 60(237):1–15, 1951.

[8] G. Governatori, F. Olivieri, A. Rotolo, and S. Scannapieco. Computing strong and weak permissions in defeasible logic. *Journal of Philosophical Logic*, 42(6):799–829, 2013.

[9] G. Sartor. *Legal Reasoning: A Cognitive Approach to the Law*. Springer, 2005.

[10] M. Hashmi, G. Governatori, H. P. Lam, and M. T. Wynn. Are we done with business process compliance: state of the art and challenges ahead. *Knowledge and Information Systems*, 57(1):79–133, 2018.

[11] T. D. Breaux and A. I. Antón. Analyzing regulatory rules for privacy and security requirements. *IEEE Transactions on Software Engineering*, 34(1):5–20, 2008.

[12] J. C. Maxwell and A. I. Antón. The production rule framework: developing a canonical set of software engineering production rules from regulatory texts. In *Proc. 20th IEEE International Requirements Engineering Conference (RE)*, pp. 51–60, 2012.

[13] M. Ehrgott. *Multicriteria Optimization*. Springer, 2nd edition, 2005.

[14] C. H. Papadimitriou and M. Yannakakis. On the approximability of trade-offs and optimal access of web sources. In *Proc. 41st Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 86–92, 2000.

[15] E. Zitzler and L. Thiele. Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation*, 3(4):257–271, 1999.

[16] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 4963, pp. 337–340. Springer, 2008.

[17] C. Barrett, A. Stump, and C. Tinelli. The SMT-LIB standard: Version 2.0. In *Proc. 8th International Workshop on Satisfiability Modulo Theories (SMT)*, 2010.

[18] Z. Fu and S. Malik. On solving the partial MAX-SAT problem. In *Proc. International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 4121, pp. 252–265. Springer, 2006.

[19] M. H. Liffiton and K. A. Sakallah. Algorithms for computing minimal unsatisfiable subsets of constraints. *Journal of Automated Reasoning*, 40(1):1–33, 2008.

[20] G. C. Necula. Proof-carrying code. In *Proc. 24th ACM Symposium on Principles of Programming Languages (POPL)*, pp. 106–119, 1997.

[21] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik. Chaff: Engineering an efficient SAT solver. In *Proc. 38th Design Automation Conference (DAC)*, pp. 530–535, 2001.

[22] J. Marques-Silva, I. Lynce, and S. Malik. Conflict-driven clause learning SAT solvers. In *Handbook of Satisfiability*, pp. 131–153. IOS Press, 2009.

[23] OneTrust LLC. OneTrust Privacy & Data Governance Cloud. https://www.onetrust.com/, accessed 2024.

[24] IBM Corporation. IBM OpenPages with Watson. https://www.ibm.com/products/openpages-with-watson, accessed 2024.

[25] ServiceNow, Inc. ServiceNow Governance, Risk, and Compliance (GRC). https://www.servicenow.com/products/governance-risk-and-compliance.html, accessed 2024.

[26] M. Veale and F. Zuiderveen Borgesius. Demystifying the Draft EU Artificial Intelligence Act. *Computer Law & Security Review*, 41:105573, 2021.

[27] Grand View Research. Governance, Risk and Compliance Market Size Report, 2024–2030. Market Research Report GVR-4-68039-895-2, 2024.

[28] D. Achlioptas. Random satisfiability. In *Handbook of Satisfiability*, pp. 245–270. IOS Press, 2009.

[29] R. Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[30] M. H. Liffiton and A. Malik. Enumerating infeasibility: Finding multiple MUSes quickly. In *Proc. International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*, LNCS 7874, pp. 160–175, 2013.

[31] Colorado General Assembly. SB 24-205: Concerning Consumer Protections for Artificial Intelligence. Signed into law, May 2024.

[32] California State Legislature. SB-1047: Safe and Secure Innovation for Frontier Artificial Intelligence Models Act. 2024.

[33] Illinois General Assembly. 740 ILCS 14 — Biometric Information Privacy Act (BIPA). 2008.

[34] The Linux Foundation. SPDX Specification v2.3. https://spdx.github.io/spdx-spec/v2.3/, 2023.

[35] International Organization for Standardization. ISO/IEC 5230:2020 — OpenChain Specification. 2020.

[36] R. K. E. Bellamy, K. Dey, M. Hind, S. C. Hoffman, S. Houde, K. Kannan, P. Lohia, J. Martino, S. Mehta, A. Mojsilović, et al. AI Fairness 360: An extensible toolkit for detecting and mitigating algorithmic bias. *IBM Journal of Research and Development*, 63(4/5):4:1–4:15, 2019.

[37] Microsoft Corporation. Responsible AI Toolbox: An open-source framework for building responsible AI solutions. https://responsibleaitoolbox.ai/, accessed 2024.

[38] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru. Model cards for model reporting. In *Proc. ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pp. 220–229, 2019.

[39] Holistic AI Ltd. Holistic AI: AI Governance Platform. https://www.holisticai.com/, accessed 2024.

[40] Credo AI Inc. Credo AI: Responsible AI Governance Platform. https://www.credo.ai/, accessed 2024.

[41] Arthur AI Inc. Arthur AI: AI Performance Monitoring. https://www.arthur.ai/, accessed 2024.

[42] TrustArc Inc. TrustArc Privacy Management Platform. https://trustarc.com/, accessed 2024.

[43] OneTrust LLC. OneTrust AI Governance: Manage AI risk and compliance. https://www.onetrust.com/products/ai-governance/, accessed 2024.

[44] National Institute of Standards and Technology. NIST AI RMF Playbook. https://airc.nist.gov/AI_RMF_Playbook, accessed 2024.

[45] European Commission. Assessment List for Trustworthy AI (ALTAI) — Self-assessment tool for the EU AI Act. https://altai.insight-centre.org/, accessed 2024.

[46] The White House. Executive Order 14110 on the Safe, Secure, and Trustworthy Development and Use of Artificial Intelligence. Federal Register, 88 FR 75191, 30 October 2023.

[47] California State Legislature. California Consumer Privacy Act (CCPA), Cal. Civ. Code §§1798.100–1798.199.100, as amended by CPRA (2020). 2018.

[48] Personal Data Protection Commission, Singapore. Personal Data Protection Act 2012 (PDPA), No. 26 of 2012, revised 2021. https://www.pdpc.gov.sg/, accessed 2024.

[49] Federative Republic of Brazil. Lei Geral de Proteção de Dados (LGPD), Law No. 13.709. 14 August 2018, effective September 2020.

[50] Parliament of Canada. Bill C-27: Digital Charter Implementation Act, 2022 — Part 3: Artificial Intelligence and Data Act (AIDA). First reading, 16 June 2022.

Table 1: AI governance frameworks encoded in RegSynth. Core frameworks (top) are fully encoded; extended frameworks (bottom) have partial encodings targeting AI-relevant provisions.

| Framework | Scope | Art. | Bind. |
|---|---|---|---|
| EU AI Act (2024/1689) | Risk-based AI regulation | 113 | Yes |
| NIST AI RMF 1.0 | AI risk management | 72 | No |
| ISO/IEC 42001 | AI management systems | 56 | Vol. |
| GDPR (inc. Art. 22) | Data protection / ADM | 99 | Yes |
| China GenAI Measures | Generative AI services | 24 | Yes |
| US EO 14110 | Federal AI governance | 38 | Yes |
| CCPA/CPRA | CA consumer privacy | 52 | Yes |
| HIPAA (AI guidance) | Health data for AI | 18 | Yes |
| Singapore PDPA | Personal data protection | 67 | Yes |
| Brazil LGPD | Data protection | 65 | Yes |
| Canada AIDA (Bill C-27) | AI & data governance | 43 | Pend. |
| UK AI Reg. Framework | Pro-innovation AI rules | 31 | No |

Table 2: RegSynth implementation statistics by crate.

| Crate | LoC | Modules | Novel |
|---|---|---|---|
| regsynth-types | 5,702 | 13 | Domain |
| regsynth-dsl | 6,662 | 12 | Yes |
| regsynth-temporal | 3,698 | 10 | Yes |
| regsynth-encoding | 3,601 | 9 | Yes |
| regsynth-solver | 5,986 | 10 | Yes |
| regsynth-pareto | 4,826 | 10 | Yes |
| regsynth-planner | 4,043 | 9 | Partial |
| regsynth-certificate | 3,631 | 8 | Yes |
| regsynth-cli | 5,582 | 19 | Standard |
| regsynth-formats | 1,764 | 5 | Standard |
| **Total** | **45,495** | **105** | |

Table 3: Comparison of RegSynth with existing compliance tools, AI fairness platforms, and governance approaches. ✓= fully supported, ∼ = partially supported, ×= not supported.

| Tool / Approach | Formal Conflict Detection | Pareto Optimization | Temporal Trajectory | Certificates | Multi-Juris. |
|---|---|---|---|---|---|
| *GRC & Compliance Platforms* | | | | | |
| OneTrust [23] | × | × | × | × | ∼ |
| IBM OpenPages [24] | × | × | × | × | ∼ |
| ServiceNow GRC [25] | × | × | × | × | ∼ |
| TrustArc [42] | × | × | × | × | ∼ |
| OneTrust AI Governance [43] | × | × | ∼ | × | ∼ |
| *AI Fairness & Responsible AI Toolkits* | | | | | |
| IBM AI Fairness 360 [36] | ∼ | × | × | ∼ | × |
| Microsoft Responsible AI [37] | ∼ | × | × | ∼ | × |
| Google Model Cards [38] | × | × | × | ∼ | × |
| *AI Governance & Monitoring Platforms* | | | | | |
| Holistic AI [39] | ∼ | × | × | ∼ | ∼ |
| Credo AI [40] | ∼ | × | × | ∼ | ∼ |
| Arthur AI [41] | × | × | ∼ | × | × |
| NIST AI RMF Playbook [44] | × | × | × | × | × |
| EU AI Act Compliance Checkers [45] | × | × | ∼ | × | × |
| *Academic / Manual Approaches* | | | | | |
| Manual Mapping | × | × | × | × | ∼ |
| Regulatory Sandboxes | × | × | × | × | × |
| LLM Assistants | ∼ | × | × | × | ∼ |
| Deontic Logic Systems [8] | ∼ | × | × | ∼ | × |
| **RegSynth** | ✓ | ✓ | ✓ | ✓ | ✓ |

Table 4: Fine-grained compliance capability comparison. Capabilities are grouped into four categories. Numbers indicate the count of supported sub-capabilities out of the group total. Dash (–) indicates the capability category is out of scope.

| Tool | Reg. Encoding (of 6) | Conflict Analysis (of 5) | Optimization (of 4) | Auditability (of 4) |
|---|---|---|---|---|
| IBM AIF360 [36] | 1 | 1 | 0 | 2 |
| Microsoft RAI [37] | 1 | 1 | 0 | 2 |
| Google Model Cards [38] | 1 | 0 | 0 | 3 |
| Holistic AI [39] | 2 | 2 | 0 | 2 |
| Credo AI [40] | 2 | 2 | 0 | 2 |
| Arthur AI [41] | 1 | 0 | 0 | 2 |
| OneTrust [23] | 3 | 0 | 0 | 3 |
| TrustArc [42] | 3 | 0 | 0 | 3 |
| Deontic Logic [8] | 4 | 3 | 0 | 1 |
| **RegSynth** | **6** | **5** | **4** | **4** |

**Reg. Encoding:** typed obligations, deontic modalities, formalizability grading, temporal intervals, cost vectors, jurisdictional composition. **Conflict Analysis:** SAT detection, MUS extraction, provenance mapping, cross-jurisdictional diagnosis, parametric thresholds. **Optimization:** multi-objective, Pareto enumeration, trajectory planning, scalarization. **Auditability:** compliance certificates, infeasibility proofs, Pareto certificates, tamper detection.

Table 5: Regulatory jurisdiction coverage comparison. ✓= obligations encoded/templated, ∼ = partial coverage, ×= not covered. RegSynth provides formal encodings; commercial tools provide checklists.

| **Tool** | ***EU AI Act*** | ***GDPR Art. 22*** | ***US EO 14110*** | ***CCPA/CPRA*** | ***HIPAA (AI)*** | ***SG PDPA*** | ***BR LGPD*** | ***CA AIDA*** | ***China GenAI*** |
|---|---|---|---|---|---|---|---|---|---|
| IBM AIF360 [36] | × | × | × | × | × | × | × | × | × |
| Microsoft RAI [37] | × | × | × | × | × | × | × | × | × |
| Google Model Cards [38] | ∼ | × | × | × | × | × | × | × | × |
| Holistic AI [39] | ✓ | ∼ | ∼ | × | × | × | × | × | × |
| Credo AI [40] | ✓ | ∼ | ∼ | ∼ | × | × | × | × | × |
| OneTrust [23] | ∼ | ✓ | ∼ | ✓ | ∼ | ∼ | ∼ | × | × |
| TrustArc [42] | ∼ | ✓ | × | ✓ | ∼ | ∼ | ∼ | × | × |
| Arthur AI [41] | ∼ | × | × | × | × | × | × | × | × |
| NIST Playbook [44] | × | × | ∼ | × | × | × | × | × | × |
| **RegSynth** | ✓ | ✓ | ✓ | ✓ | ∼ | ✓ | ✓ | ∼ | ✓ |